

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**

KHOA TOÁN - TIN HỌC



QUY HOẠCH TUYỂN TÍNH

VIẾT CHƯƠNG TRÌNH GIẢI BÀI TOÁN QUY HOẠCH TUYỂN TÍNH TỔNG QUÁT

Thành phố Hồ Chí Minh, ngày 10 tháng 6 năm 2026

Mục lục

1	Giới thiệu	2
2	Hướng dẫn sử dụng	3
2.1	Biên dịch và chạy chương trình	3
2.2	Luồng nhập dữ liệu đầu vào	3
2.3	Chi tiết từng bước nhập liệu	4
2.4	Chọn phương pháp giải	5
2.5	Đọc và hiểu kết quả đầu ra	6
2.6	Ví dụ minh hoạ đầu vào	7
2.7	Bảng tra cứu nhanh ký hiệu	11
2.8	Xử lý lỗi thường gặp	12
3	Ưu điểm và khuyết điểm	13
3.1	Ưu điểm	13
3.2	Khuyết điểm	13
4	Các phần mềm giải bài toán QHTT khác	14
4.1	LINGO	14
4.2	IBM ILOG CPLEX	15
4.3	Gurobi Optimizer	17
4.4	GLPK (GNU Linear Programming Kit)	18
4.5	MATLAB Optimization Toolbox	19
4.6	OR-Tools của Google	21
5	Tài liệu tham khảo	23

1. Giới thiệu

Báo cáo này trình bày quá trình xây dựng và triển khai một chương trình máy tính bằng ngôn ngữ C++ nhằm giải quyết bài toán Quy hoạch tuyến tính (QHTT) ở dạng tổng quát. Chương trình được thiết kế linh hoạt, cho phép tiếp nhận đầu vào là các mô hình toán học với hàm mục tiêu cần tối ưu hóa (Min/Max), đa dạng các loại dấu ràng buộc ($\geq$, $\leq$, $=$) cũng như các điều kiện khắt khe của biến (biến không âm, biến không dương, hoặc biến tự do). Từ mô hình tổng quát ban đầu, chương trình sẽ tự động chuyển đổi sang dạng chuẩn tắc và tiến hành tìm nghiệm thông qua các thuật toán giải tích kinh điển.

Điểm nổi bật của chương trình không chỉ dừng lại ở việc xuất ra nghiệm tối ưu cuối cùng, mà còn nằm ở khả năng minh họa tường minh quá trình tính toán. Người dùng có thể theo dõi chi tiết các bước chuyển đổi mô hình, cũng như sự biến đổi của các bảng đơn hình (từ vệt) qua từng vòng lặp. Chức năng này biến chương trình trở thành một công cụ hỗ trợ đắc lực, giúp người dùng tiết kiệm thời gian tính toán thủ công, hạn chế sai sót và củng cố tư duy nền tảng về thuật toán. Để đảm bảo tính toàn vẹn và khả năng xử lý mọi trường hợp của bài toán QHTT, chương trình đã được cài đặt tích hợp các phương pháp sau:

- Phương pháp Đơn hình chuẩn (Standard Simplex Method): Thuật toán nền tảng, được áp dụng để tìm phương án tối ưu khi bài toán đã có sẵn một phương án cực biên cơ sở khả thi (tất cả các hệ số vế phải $b_i \geq 0$).
- Quy tắc Bland (Bland's Rule): Một tinh chỉnh quan trọng của phương pháp Đơn hình nhằm giải quyết triệt để hiện tượng lặp vô hạn (xoay vòng) trong các trường hợp bài toán bị suy biến.
- Phương pháp Hai pha (Two-Phase Method): Thuật toán tổng quát và mạnh mẽ nhất, cho phép giải quyết mọi dạng bài toán QHTT. Phương pháp này đặc biệt thiết yếu khi mô hình không có sẵn phương án cơ sở ban đầu (tồn tại hệ số $b_i < 0$ ở dạng chuẩn tắc), đòi hỏi phải bổ sung các biến giả tạo để tìm nghiệm khả thi ở Pha 1 trước khi tối ưu hóa ở Pha 2.

2. Hướng dẫn sử dụng

2.1 Biên dịch và chạy chương trình

Chương trình được viết bằng C++11. Sử dụng trình biên dịch g++ hoặc bất kỳ trình biên dịch C++ nào tương thích.

Trên Linux / macOS

```
g++ -std=c++11 -o qhtt main.cpp ./qhtt
```

Trên Windows (MinGW)

```
g++ -o qhtt.exe main.cpp qhtt.exe
```

*Lưu ý: Trên VS Code, nhấn Ctrl+Shift+B để build nhanh nếu đã cấu hình tasks.json.
Đảm bảo file mã nguồn lưu với encoding UTF-8 để hiển thị đúng tiếng Việt.*

2.2 Luồng nhập dữ liệu đầu vào

Sau khi chạy chương trình, lần lượt nhập các thông tin theo đúng thứ tự sau:

Bước	Thông tin cần nhập	Giá trị hợp lệ
1	Loại tối ưu	1 (Min) hoặc 2 (Max)
2	Số biến quyết định	Số nguyên dương (>0)
3	Số ràng buộc	Số nguyên dương (>0)
4	Hệ số hàm mục tiêu $c_1, c_2, \dots$	Số thực (âm, dương, hoặc 0)
5	Hệ số các ràng buộc $+b_i +$ dấu	Số thực; dấu: 1/2/3
6	Điều kiện dấu từng biến	1 (≥ 0), 2 (≤ 0), hoặc 3 (tự do)
7	Phương pháp giải	1, 2, 3 hoặc 4

Bảng 2.1: Luồng nhập dữ liệu

Lưu ý: Chương trình tự kiểm tra đầu vào ở mỗi bước. Nếu nhập sai (chữ thay số, ngoài khoảng hợp lệ), chương trình sẽ yêu cầu nhập lại ngay.

2.3 Chi tiết từng bước nhập liệu

Bước 1. Loại tối ưu

- Nhập 1: Bài toán tối thiểu hoá (Min z)
- Nhập 2: Bài toán tối đa hoá (Max z)

Lưu ý: Bài toán Max được tự động chuyển thành -Min nội bộ. Hệ số hàm mục tiêu sẽ bị đổi dấu trong mô hình chuẩn tắc — đây là bình thường.

Bước 2 & 3. Số biến và số ràng buộc

- Số biến: đếm số biến $x_1, x_2, \dots, x_n$ trong bài toán gốc.
- Số ràng buộc: đếm số bất đẳng thức/phương trình (không kể điều kiện dấu riêng lẻ của từng biến).

Lưu ý: Ràng buộc dạng '=' sẽ được tách thành 2 ràng buộc ($\leq$ và $\geq$) trong mô hình chuẩn tắc, làm tăng số ràng buộc hiển thị.

Bước 4. Hệ số hàm mục tiêu

- Nhập hệ số $c_1, c_2, \dots, c_n$ lần lượt, mỗi giá trị nhấn Enter.
- Chấp nhận số thực: ví dụ 4, -2.5, 0.

Bước 5. Hệ số các ràng buộc

Với mỗi ràng buộc thứ j , nhập theo thứ tự:

1. Hệ số $a_{j1}, a_{j2}, \dots$ cho từng biến (mỗi giá trị nhấn Enter).
2. Giá trị vế phải b_i .
3. Dấu của ràng buộc: 1 ($\geq$), 2 ($\leq$), 3 (=).

Dấu nhập	Ý nghĩa	Ghi chú
1	$\geq$ (lớn hơn hoặc bằng)	Chương trình đổi thành $\leq$ bằng cách nhân -1
2	$\leq$ (nhỏ hơn hoặc bằng)	Dạng chuẩn, không cần xử lý thêm
3	= (bằng)	Tách thành 2 ràng buộc: $\leq$ và $\geq$

Bảng 2.2: Dấu ràng buộc

Bước 6. Điều kiện dấu từng biến

Giá trị	Ý nghĩa	Xử lý nội bộ
1	$x_i \geq 0$ (không âm — phổ biến nhất)	Giữ nguyên biến x_i
2	$x_i \leq 0$ (không dương)	Thay $x_i = -y_i, y_i \geq 0$
3	x_i tự do (không hạn chế dấu)	Tách $x_i = x_i^+ - x_i^-$, cả hai ≥ 0

Bảng 2.3: Điều kiện dấu biến

Lưu ý: Biến tự do (loại 3) làm tăng số biến lên 2 (x_i^+ và x_i^-). Số biến trong mô hình chuẩn tắc có thể lớn hơn số biến gốc.

2.4 Chọn phương pháp giải

Lựa chọn	Phương pháp	Điều kiện áp dụng	Khi nào nên dùng
1	Phương pháp Đơn hình	Tất cả $b_i \geq 0$	Bài toán chuẩn, nhanh nhất
2	Phương pháp Bland	Tất cả $b_i \geq 0$	Tránh cycling, an toàn hơn đơn hình
3	Phương pháp Hai pha	Mọi trường hợp	Khi có $b_i < 0$ hoặc không chắc nghiệm khả thi tồn tại
4	Tất cả phương pháp	Áp dụng tùy TH	So sánh kết quả giữa các phương pháp

Bảng 2.4: Các phương pháp giải

Lưu ý: Nếu không chắc chắn, hãy chọn 3 (Hai pha). Phương pháp này áp dụng được mọi trường hợp và tự xử lý nghiệm khả thi ở Pha 1 trước khi tối ưu ở Pha 2.

Sự khác biệt giữa Đơn hình và Bland

- **Đơn hình:** Chọn biến vào là cột có $c_j - z_j$ âm nhất (nhỏ nhất). Nhanh hơn nhưng có thể bị lặp vô hạn (cycling).
- **Bland:** Chọn biến vào là cột có chỉ số nhỏ nhất trong số các cột có $c_j - z_j$ âm. Đảm bảo hội tụ, tránh cycling.

2.5 Đọc và hiểu kết quả đầu ra

Sau khi nhập đủ dữ liệu, chương trình in kết quả theo thứ tự:

1. Mô hình tổng quát (như bạn đã nhập)
2. Mô hình chuẩn tắc (sau khi chuẩn hoá)
3. Dữ liệu chi tiết mô hình chuẩn tắc (tên biến, ma trận A, vector b, vector c)
4. Quá trình giải từng bước (các lần lặp)
5. Kết luận nghiệm

Các loại kết luận có thể nhận được

Kết luận	Ý nghĩa	Thông tin in ra
Nghiệm tối ưu DUY NHẤT	Bài toán có đúng một nghiệm	Giá trị z^* và từng biến x_i
VÔ SỐ NGHIỆM (1 biến tự do)	Tập nghiệm là một đoạn thẳng hoặc một tia	Biểu thức tham số + khoảng hợp lệ $[a, b]$
VÔ SỐ NGHIỆM (nhiều biến tự do)	Tập nghiệm là mặt phẳng/đa tạp	Biểu thức theo các biến tự do
Vô nghiệm	Miền khả thi rỗng	Thông báo: Bài toán vô nghiệm
Không giới nội	$z \rightarrow \pm\infty$	Thông báo $z = -\infty$ (Min) hoặc $+\infty$ (Max)

Bảng 2.5: Kết luận nghiệm

Ký hiệu trong quá trình giải

Ký hiệu	Ý nghĩa
$x_1, x_2, \dots$	Biến quyết định ban đầu
x_1^+, x_1^-	Biến thay thế cho biến tự do x_1 ($x_1 = x_1^+ - x_1^-$)
$y_1, y_2, \dots$	Biến thay thế cho biến ≤ 0 ($x_i = -y_i$)
$s_1, s_2, \dots$	Biến bù (slack variable) thêm vào ràng buộc $\leq$

x_0	Biến phụ đặc biệt trong Pha 1 của phương pháp hai pha
z_j	Giá trị hàng z tại cột j: tổng (cb * cột j)
$c_j - z_j$	Hàng mục tiêu: âm = chưa tối ưu, tất cả ≥ 0 = đã tối ưu
Biến vào	Biến được đưa vào cơ sở (cột pivot)
Biến ra	Biến bị đưa ra khỏi cơ sở (hàng pivot)
Tỉ lệ tại ràng buộc i	Ratio test: b_i / a_{ij} để chọn hàng pivot

Bảng 2.6: Ký hiệu trong quá trình giải

2.6 Ví dụ minh hoạ đầu vào

Bài toán

$$\text{Min } z = -3x_1 - x_2$$

$$\text{Ràng buộc: } -x_1 + x_2 \geq 1$$

$$x_1 + x_2 \geq 3$$

$$2x_1 + x_2 \leq 4$$

$$\text{Điều kiện: } x_1, x_2 \geq 0$$

Nhập liệu mẫu

Nhap loại toi uu (1 cho min, 2 cho max): 1

Nhap so bien (xi): 2

Nhap so rang buoc: 3

Ham muc tieu:

He so cho x1: -3

He so cho x2: -1

Rang buoc thu 1:

He so x1: -1

He so x2: 1

He so thuc (bi) cua rang buoc: 1

Dau cua rang buoc 1(1: >=, 2: <=, 3: =): 1

Rang buoc thu 2:

He so x1: 1

He so x2: 1

He so thuc (bi) cua rang buoc: 3

Dau cua rang buoc 2(1: >=, 2: <=, 3: =): 1

Rang buoc thu 3:

He so x1: 2

He so x2: 1

He so thuc (bi) cua rang buoc: 4

Dau cua rang buoc 3(1: >=, 2: <=, 3: =): 2

Dieu kien cho x1 (1: >=, 2: <=, 3: tu do): 1

Dieu kien cho x2 (1: >=, 2: <=, 3: tu do): 1

Chon Phuong phap giai bai toan QHTT (1: Don hinh, 2: Bland, 3: 2 pha, 4: Tat ca)
: 4

Kết quả đầu ra từ chương trình

=====

Mo hinh tong quat

=====

Min -3x1 + -1x2

-1x1 + 1x2 >= 1

1x1 + 1x2 >= 3

2x1 + 1x2 <= 4

x1 >= 0

x2 >= 0

=====

Mo hinh chuan tac

=====

Min -3x1 - 1x2

1x1 - 1x2 <= -1

-1x1 - 1x2 <= -3

2x1 + 1x2 <= 4

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Dữ liệu của mô hình chuẩn tắc khi đã chuẩn hóa:

Tổng số biến mới: 2

Tổng số ràng buộc mới: 3

Tên các biến sau chuẩn hóa (x^T): [x_1 x_2]

Hệ số hàm mục tiêu (c^T): [-3 -1]

Mã trận hệ số A:

```
[ 1 -1
  -1 -1
   2 1 ]
```

Vector b: [-1 -3 4]

Không thể áp dụng phương pháp đơn hình vì tồn tại số thực $b_i < 0$

Không thể áp dụng phương pháp Bland vì tồn tại số thực $b_i < 0$

Phương pháp hai pha:

===== PHA 1 =====

Lần lặp 0 (Pha 1)

$$zeta = x_0$$

$$x_2 = -1.00 - 1.00x_1 + 1.00x_0$$

$$s_2 = -3.00 + 1.00x_1 + 1.00x_0$$

$$s_3 = 4.00 - 2.00x_1 - 1.00s_2 + 1.00x_0$$

Chọn biến vào: x_0

$$\text{Ti lệ tại ràng buộc 1: } -1.00/1.00 = -1.00$$

$$\text{Ti lệ tại ràng buộc 2: } -3.00/1.00 = -3.00$$

$$\text{Ti lệ tại ràng buộc 3: } 4.00/1.00 = 4.00$$

Chọn biến ra: w_2 vì số thực b âm nhất

Lần lặp 1 (Pha 1)

$$zeta = 3.00 - 1.00x_1$$

$$x_2 = 2.00 - 2.00x_1 - 1.00s_1$$

$$x_0 = 3.00 - 1.00x_1$$

$$s_3 = 7.00 - 3.00x_1 - 1.00s_1$$

Chọn biến vào: x_1

Tỉ lệ tại ràng buộc 1: $2.00/2.00 = 1.00$

Tỉ lệ tại ràng buộc 2: $3.00/1.00 = 3.00$

Tỉ lệ tại ràng buộc 3: $7.00/3.00 = 2.33$

Chọn biến ra: x_2

Lần lặp 2 (Pha 1)

$zeta = 2.00 - 1.00x_2 + 0.50s_1$

$x_1 = 1.00 - 0.50s_1$

$x_0 = 2.00 - 1.00x_2 + 0.50s_1$

$s_3 = 4.00 - 2.00x_2 + 1.50s_1 - 1.00s_3$

Chọn biến vào: x_2

Tỉ lệ tại ràng buộc 1: không xác định

Tỉ lệ tại ràng buộc 2: $2.00/1.00 = 2.00$

Tỉ lệ tại ràng buộc 3: $4.00/2.00 = 2.00$

Chọn biến ra: x_0

Lần lặp 3 (Pha 1)

$zeta = 0.00 + 1.00x_0$

$x_1 = 1.00 - 0.50s_1$

$x_2 = 2.00 + 0.50s_1 - 1.00x_0$

$s_3 = 0.00 + 0.50s_1 - 1.00s_3 + 2.00x_0$

KẾT LUẬN PHA 1: Thỏa. Tiếp tục vào Pha 2.

===== PHA 2 =====

Lần lặp 0 (Pha 2)

$z' = -3.00 - 1.00x_2 + 1.00s_3$

$x_1 = 1.00 - 0.33s_3$

$s_1 = 2.00 - 1.00x_2 - 0.33s_3$

$s_2 = 0.00 - 0.67s_3$

Biến vào: x_2

Tỉ lệ tại ràng buộc 1: không xác định

Tỉ lệ tại ràng buộc 2: $2.00/1.00 = 2.00$

Tỉ lệ tại ràng buộc 3: không xác định

Biến ra: s_1

Lần lặp 1 (Pha 2)

$z' = -5.00 + 0.33s_1 + 1.33s_3$

$x_1 = 1.00 - 0.33s_1 - 0.33s_3$

$$x_2 = 2.00 + 0.67s_1 - 0.33s_3$$

$$s_2 = 0.00 + 0.33s_1 - 0.67s_3$$

KET LUAN: Bài toán có NGHIỆM tối ưu DUY NHẤT!

Giá trị hàm mục tiêu $z = -5.00$

Nghiệm tối ưu:

$$x_1 = 1.00$$

$$x_2 = 2.00$$

Phân tích kết quả

- Do tồn tại ràng buộc dạng $\geq$ nên sau khi chuẩn hóa xuất hiện $b_i < 0$, do đó phương pháp Đơn hình và Bland không áp dụng được.
- Chương trình tự động chuyển sang **Phương pháp Hai pha**.
- Pha 1: Đưa biến phụ x_0 về 0, kết luận bài toán có phương án khả thi.
- Pha 2: Thực hiện tối ưu, sau 2 lần lặp đạt nghiệm tối ưu.
- **Nghiệm tối ưu DUY NHẤT:** $x_1 = 1.00, x_2 = 2.00$
- **Giá trị tối ưu:** $z = -5.00$

2.7 Bảng tra cứu nhanh ký hiệu

Ký hiệu	Loại	Ý nghĩa đầy đủ
Min / -Min	Hàm mục tiêu	Bài toán tối thiểu hoá (Max đổi sang -Min nội bộ)
x_i	Biến chính	Biến quyết định thứ i ($i = 1, 2, \dots, n$)
x_i^+, x_i^-	Biến thay thế	Tách biến tự do: $x_i = x_i^+ - x_i^-$, cả hai ≥ 0
y_i	Biến đổi dấu	Thay biến ≤ 0 : $x_i = -y_i, y_i \geq 0$
s_i	Biến bù	Slack variable thêm vào để chuyển $\leq$ thành $=$
x_0	Biến pha 1	Biến phụ đặc biệt, Min x_0 trong Pha 1 của hai pha
b_i	Vế phải	Hệ số số thực ở vế phải ràng buộc thứ i
c_j	Hàm mục tiêu	Hệ số biến x_j trong hàm mục tiêu z
z_j	Hàng z	Giá trị tính được tại cột j: $\sum (c_b \cdot a_{ij})$

$c_j - z_j$	Hàng mục tiêu	Chỉ tiêu tối ưu; tất cả $\geq 0 \rightarrow$ nghiệm tối ưu
Lần lặp k	Vòng lặp	Bước xoay thứ k trong bảng đơn hình
ζ (zeta)	Pha 1	Hàm mục tiêu của Pha 1 ($= x_0$)

Bảng 2.7: Bảng tra cứu ký hiệu

2.8 Xử lý lỗi thường gặp

Thông báo / Tình huống	Nguyên nhân	Cách xử lý
Không thể áp dụng phương pháp đơn hình	Tồn tại $b_i < 0$ sau khi chuẩn hoá (ràng buộc $\geq$ đổi thành $\leq$)	Chuyển sang phương pháp 3 (Hai pha)
Không thể áp dụng phương pháp Bland	Tương tự đơn hình: tồn tại $b_i < 0$	Chuyển sang phương pháp 3 (Hai pha)
Bài toán vô nghiệm	Pha 1 không đưa được x_0 về 0 $\rightarrow$ miền khả thi rỗng (ràng buộc mâu thuẫn nhau)	Kiểm tra lại các ràng buộc; có thể chúng mâu thuẫn
Bài toán không giới nội ($z = \pm\infty$)	Tìm được biến vào nhưng không có tỉ lệ dương $\rightarrow$ không có biến ra	Kiểm tra mô hình; bài toán Max có thể thiếu ràng buộc trên
Sau k lần xoay, đã quay lại từ vựng gốc	Đơn hình bị cycling: bảng lặp lại trạng thái ban đầu	Chuyển sang Bland (2) hoặc Hai pha (3)
Đã đạt số lần lặp tối đa (20)	Bài toán không hội tụ trong 20 bước — thường do cycling	Thử phương pháp Bland hoặc Hai pha; kiểm tra lại số liệu
Chương trình yêu cầu nhập lại	Nhập sai kiểu (chữ thay số) hoặc ngoài khoảng hợp lệ	Nhập đúng giá trị theo hướng dẫn trong dấu ngoặc đơn

Bảng 2.8: Xử lý lỗi thường gặp

Lưu ý: Chương trình dùng sai số $\epsilon = 1e-9$ cho so sánh số thực. Với bài toán có hệ số rất lớn ($> 10^6$), kết quả có thể bị ảnh hưởng bởi sai số làm tròn kiểu double.

3. Ưu điểm và khuyết điểm

3.1 Ưu điểm

- Hỗ trợ nhiều thuật toán QHTT, cho phép giải quyết các dạng bài toán khác nhau.
- Xử lý được các trường hợp đặc biệt như biến tự do, biến có dấu âm, và các loại ràng buộc phức tạp. Có khả năng phát hiện các trường hợp vô nghiệm, không giới nội, vô số nghiệm, lặp vô hạn hoặc quay lại từ vệt gốc.
- Việc in ra chi tiết từng bước của thuật toán bao gồm việc chọn biến vào, biến ra, tỉ lệ tại các ràng buộc và ràng buộc giúp người dùng dễ dàng theo dõi và kiểm tra lại quá trình giải bằng tay của mình.
- Có cơ chế kiểm tra và yêu cầu nhập lại khi người dùng nhập sai định dạng (ví dụ: nhập chữ thay vì số), giúp chương trình hoạt động ổn định.

3.2 Khuyết điểm

- Việc nhập dữ liệu hoàn toàn bằng văn bản có thể trở nên chậm và dễ nhầm lẫn đối với các bài toán có nhiều biến và ràng buộc. Nếu xảy ra sai sót trong quá trình nhập liệu, người dùng buộc phải khởi động lại toàn bộ chương trình.
- Chương trình sử dụng con trỏ thô và cấp phát động (new/delete[]). Mặc dù đã được giải phóng đúng cách nhưng vẫn có thể dễ gây ra lỗi rò rỉ bộ nhớ trong các dự án lớn.
- Thuật toán được cài đặt cho mục đích giải các bài toán trong sách, không được tối ưu về hiệu năng cũng như không có đủ testcase để thử tất cả các trường hợp.
- Khuyết điểm về biểu diễn nghiệm trong trường hợp vô số nghiệm phức tạp: Trong trường hợp bài toán có vô số nghiệm với hai hay nhiều biến tự do, chương trình hiện tại chỉ có thể đưa ra điều kiện của biến tự do là các ràng buộc chứa nó lớn hơn hoặc bằng không.

4. Các phần mềm giải bài toán QHTT khác

Hiện nay, cùng với sự phát triển của công nghệ thông tin, nhiều phần mềm và công cụ tối ưu hóa đã được xây dựng nhằm hỗ trợ giải quyết các bài toán quy hoạch tuyến tính một cách nhanh chóng và hiệu quả. Các phần mềm này không chỉ giúp tìm nghiệm tối ưu mà còn hỗ trợ mô hình hóa bài toán, phân tích độ nhạy và xử lý các mô hình có quy mô lớn. Trong số đó, LINGO, IBM ILOG CPLEX và Gurobi Optimizer, GNU Linear Programming Kit, MATLAB Optimization Toolbox, OR-Tools là những phần mềm được sử dụng phổ biến trong giảng dạy, nghiên cứu và thực tiễn sản xuất.

4.1 LINGO

Giới thiệu

LINGO được phát triển bởi LINDO Systems. Điểm đặc trưng nhất của LINGO là nó sở hữu một ngôn ngữ mô hình hóa rất mạnh mẽ, cho phép người dùng nhập các mô hình tuyến tính, phi tuyến hay nguyên một cách trực quan bằng các ký hiệu tổng đại số ($\sum$) gần như tương đồng với cách viết trên giấy, giúp rút ngắn thời gian làm bài với những bài toán có hàng trăm dòng ràng buộc.

Khả năng giải

- Quy hoạch tuyến tính (LP)
- Quy hoạch nguyên (IP)
- Quy hoạch hỗn hợp nguyên (MILP)
- Quy hoạch phi tuyến (NLP)
- Quy hoạch bậc hai (QP)
- Quy hoạch ngẫu nhiên

Ưu điểm

LINGO có giao diện đơn giản, dễ sử dụng và phù hợp với sinh viên cũng như người mới học quy hoạch tuyến tính. Ngôn ngữ mô hình hóa của phần mềm gần với cách biểu diễn toán học thông thường, giúp việc xây dựng mô hình trở nên thuận tiện. Ngoài ra, LINGO còn cung cấp nhiều tài liệu và ví dụ mẫu hỗ trợ quá trình học tập và nghiên cứu.

Nhược điểm

Phiên bản miễn phí của LINGO bị giới hạn về kích thước bài toán, do đó chưa phù hợp với các mô hình có quy mô lớn. Bên cạnh đó, hiệu năng xử lý của phần mềm đối với những bài toán phức tạp thường thấp hơn so với các bộ giải chuyên nghiệp như CPLEX hoặc Gurobi.

Phiên bản hiện hành

- **Phiên bản: LINGO 22.0.** Phiên bản này cải tiến mạnh mẽ tốc độ giải thuật, tăng cường liên kết dữ liệu trực tiếp với Excel và hỗ trợ extension chỉnh sửa/chạy mô hình ngay trong VS Code.

Hướng dẫn sử dụng

Người dùng có thể trực tiếp nhập hàm mục tiêu và các ràng buộc thông qua ngôn ngữ mô hình hóa được tích hợp sẵn trong phần mềm. Sau khi hoàn thành việc xây dựng mô hình, chương trình sẽ tự động thực hiện quá trình tính toán và trả về nghiệm tối ưu cùng các thông tin hỗ trợ phân tích.

Bước 1: Cài đặt và khởi động phần mềm LINGO trên máy tính.

Bước 2: Tạo một mô hình mới bằng cách chọn chức năng tạo tệp mới trong giao diện làm việc của phần mềm.

Bước 3: Nhập hàm mục tiêu và các ràng buộc của bài toán theo cú pháp của LINGO.

Bước 4: Thực hiện lệnh giải bài toán bằng chức năng Solve.

Bước 5: Phân tích kết quả tối ưu do phần mềm cung cấp, bao gồm giá trị hàm mục tiêu, giá trị các biến quyết định và các thông tin phân tích liên quan.

4.2 IBM ILOG CPLEX

Giới thiệu

IBM ILOG CPLEX Optimization Studio là một trong những phần mềm tối ưu hóa mạnh nhất hiện nay, được phát triển bởi IBM. Đây là công cụ được sử dụng rộng rãi trong các doanh nghiệp lớn, đặc biệt trong các lĩnh vực logistics, vận tải, lập kế hoạch sản xuất, quản lý chuỗi cung ứng và tài chính.

Khả năng giải

- Quy hoạch tuyến tính (LP)

- Quy hoạch hỗn hợp nguyên (MILP)
- Quy hoạch bậc hai (QP)
- Quy hoạch ràng buộc bậc hai (QCP)
- Quy hoạch hình nón bậc hai (SOCP)
- Các bài toán tối ưu quy mô rất lớn

Thuật toán sử dụng

- Simplex
- Dual Simplex
- Interior Point (Barrier)
- Branch and Bound
- Branch and Cut

Ưu điểm

IBM ILOG CPLEX nổi bật với tốc độ xử lý nhanh và độ ổn định cao khi giải các bài toán tối ưu hóa có quy mô lớn. Phần mềm hỗ trợ nhiều ngôn ngữ lập trình như Python, Java, C++, C và MATLAB, giúp người dùng dễ dàng tích hợp vào các ứng dụng thực tế và hệ thống doanh nghiệp.

Nhược điểm

Mặc dù sở hữu hiệu năng mạnh mẽ, IBM ILOG CPLEX có chi phí bản quyền tương đối cao và yêu cầu người dùng có kiến thức nhất định về tối ưu hóa cũng như lập trình. Vì vậy, phần mềm có thể gây khó khăn cho những người mới bắt đầu tiếp cận lĩnh vực này.

Phiên bản hiện hành

- **Phiên bản: IBM ILOG CPLEX 22.1.1** (được cập nhật liên tục với khả năng tối ưu hóa điện toán đám mây và cải tiến mạnh cho quy hoạch nguyên hỗn hợp MIP).

Hướng dẫn sử dụng

Người dùng xây dựng mô hình bằng ngôn ngữ OPL hoặc thông qua các ngôn ngữ lập trình như Python, Java, C++ và C#. Sau khi mô hình được thiết lập, bộ giải CPLEX sẽ

thực hiện quá trình tối ưu hóa và cung cấp các kết quả chi tiết liên quan đến nghiệm tối ưu của bài toán.

Bước 1: Cài đặt và khởi động phần mềm IBM ILOG CPLEX Optimization Studio.

Bước 2: Tạo một dự án mới và xây dựng mô hình tối ưu hóa bằng ngôn ngữ OPL hoặc ngôn ngữ lập trình được hỗ trợ.

Bước 3: Khai báo hàm mục tiêu, các biến quyết định và các ràng buộc của bài toán.

Bước 4: Thực hiện lệnh chạy chương trình để bộ giải CPLEX tiến hành tối ưu hóa.

Bước 5: Quan sát và phân tích kết quả tối ưu được hiển thị trong cửa sổ kết quả của phần mềm.

4.3 Gurobi Optimizer

Giới thiệu

Gurobi Optimizer là một bộ giải tối ưu thương mại được đánh giá là một trong những công cụ mạnh nhất hiện nay đối với các bài toán quy hoạch tuyến tính và quy hoạch nguyên. Phần mềm được phát triển bởi Gurobi Optimization LLC và được sử dụng rộng rãi trong nghiên cứu học thuật cũng như các ứng dụng công nghiệp.

Khả năng giải

- Quy hoạch tuyến tính (LP)
- Quy hoạch hỗn hợp nguyên (MILP)
- Quy hoạch bậc hai (QP)
- Quy hoạch nguyên bậc hai hỗn hợp (MIQP)
- Quy hoạch ràng buộc bậc hai (QCP)
- Quy hoạch hỗn hợp nguyên có ràng buộc bậc hai (MIQCP)

Ưu điểm

Gurobi Optimizer được đánh giá cao nhờ tốc độ xử lý nhanh và khả năng giải hiệu quả các bài toán tối ưu hóa có quy mô lớn. Phần mềm tận dụng tốt sức mạnh của các bộ vi xử lý đa lõi, đồng thời cung cấp API mạnh mẽ, đặc biệt là đối với ngôn ngữ Python. Ngoài ra, Gurobi còn sở hữu hệ thống tài liệu hướng dẫn đầy đủ, giúp người dùng dễ dàng tiếp cận và khai thác các tính năng của phần mềm.

Nhược điểm

Mặc dù có hiệu năng vượt trội, Gurobi Optimizer là phần mềm thương mại với chi phí bản quyền tương đối cao. Bên cạnh đó, việc sử dụng hiệu quả các tính năng của phần mềm thường đòi hỏi người dùng phải có kiến thức về lập trình và tối ưu hóa.

Phiên bản hiện hành

- **Phiên bản: Gurobi 13.0.** Phiên bản này tối ưu sâu cho kiến trúc chip đa nhân, nâng cấp thuật toán tính toán song song giúp xử lý các mô hình tuyến tính nguyên hỗn hợp (MIP) nhanh hơn vượt trội.

Hướng dẫn sử dụng

Gurobi Optimizer chủ yếu được sử dụng thông qua các thư viện lập trình, đặc biệt là Python. Người dùng cần xây dựng mô hình bằng cách khai báo các biến quyết định, hàm mục tiêu và hệ thống ràng buộc, sau đó sử dụng bộ giải của Gurobi để tìm nghiệm tối ưu.

Bước 1: Cài đặt phần mềm Gurobi và kích hoạt giấy phép sử dụng.

Bước 2: Cài đặt thư viện Gurobi cho ngôn ngữ lập trình được lựa chọn, phổ biến nhất là Python.

Bước 3: Xây dựng mô hình bằng cách khai báo các biến quyết định, hàm mục tiêu và các ràng buộc.

Bước 4: Sử dụng bộ giải của Gurobi để thực hiện quá trình tối ưu hóa.

Bước 5: Trích xuất và phân tích nghiệm tối ưu thu được từ chương trình.

4.4 GLPK (GNU Linear Programming Kit)

Giới thiệu

GLPK (GNU Linear Programming Kit) là một bộ công cụ tối ưu hóa mã nguồn mở được phát triển bởi Dự án GNU. Phần mềm được thiết kế để giải các bài toán quy hoạch tuyến tính (Linear Programming - LP), quy hoạch nguyên (Integer Programming - IP) và quy hoạch hỗn hợp nguyên (Mixed Integer Programming - MIP).

Khả năng giải

- Quy hoạch tuyến tính (LP)
- Quy hoạch hỗn hợp nguyên (MILP)

Ưu điểm

GLPK là một phần mềm hoàn toàn miễn phí và mã nguồn mở, giúp người dùng dễ dàng tiếp cận và sử dụng mà không phát sinh chi phí bản quyền. Bên cạnh đó, phần mềm có khả năng tích hợp với nhiều ngôn ngữ lập trình và các dự án nghiên cứu, phù hợp cho mục đích học tập và phát triển các ứng dụng tối ưu hóa.

Nhược điểm

So với các bộ giải thương mại như CPLEX và Gurobi, tốc độ xử lý của GLPK thường chậm hơn khi giải các bài toán có quy mô lớn. Ngoài ra, giao diện của phần mềm còn khá đơn giản và chưa thực sự thân thiện đối với người dùng mới.

Phiên bản hiện hành

- **Phiên bản: GLPK 5.0.** Phiên bản này tập trung vào việc sửa lỗi, tăng cường độ ổn định của thuật toán đơn hình hai pha (Two-phase simplex method) và thuật toán điểm trong.

Hướng dẫn sử dụng

Mô hình thường được xây dựng bằng ngôn ngữ MathProg hoặc thông qua các thư viện hỗ trợ trong các ngôn ngữ lập trình. Sau khi nhập dữ liệu và mô tả bài toán, bộ giải GLPK sẽ thực hiện việc tính toán và xuất kết quả tối ưu.

Bước 1: Cài đặt bộ công cụ GLPK trên hệ thống.

Bước 2: Xây dựng mô hình toán học bằng ngôn ngữ MathProg hoặc thông qua các thư viện hỗ trợ.

Bước 3: Nhập dữ liệu và các tham số cần thiết cho bài toán.

Bước 4: Thực hiện lệnh chạy bộ giải GLPK để tìm nghiệm tối ưu.

Bước 5: Đọc và phân tích kết quả được xuất ra từ chương trình.

4.5 MATLAB Optimization Toolbox

Giới thiệu

MATLAB Optimization Toolbox là bộ công cụ tối ưu hóa được phát triển bởi MathWorks và được tích hợp trong môi trường tính toán MATLAB. Đây là một công cụ mạnh mẽ, được sử dụng phổ biến trong các trường đại học, viện nghiên cứu và các ngành kỹ thuật nhờ khả năng kết hợp giữa tối ưu hóa, tính toán số và trực quan hóa dữ liệu.

Khả năng giải

- Quy hoạch tuyến tính (LP)
- Quy hoạch hỗn hợp nguyên (MILP)
- Quy hoạch bậc hai (QP)
- Quy hoạch phi tuyến (NLP)
- Bài toán bình phương tối thiểu
- Tối ưu hoá đa mục tiêu

Ưu điểm

MATLAB Optimization Toolbox được tích hợp trực tiếp trong môi trường MATLAB, giúp người dùng dễ dàng kết hợp giữa tối ưu hóa, tính toán số và xử lý dữ liệu. Công cụ này hỗ trợ hiệu quả việc trực quan hóa kết quả thông qua các đồ thị và biểu đồ, đồng thời rất phù hợp cho hoạt động nghiên cứu khoa học, giảng dạy và các ứng dụng kỹ thuật.

Nhược điểm

Để sử dụng MATLAB Optimization Toolbox, người dùng cần có bản quyền MATLAB, dẫn đến chi phí sử dụng tương đối cao. Ngoài ra, đối với một số bài toán tối ưu hóa có quy mô rất lớn, hiệu năng xử lý của công cụ này thường không cao bằng các bộ giải chuyên dụng như Gurobi hoặc CPLEX.

Phiên bản hiện hành

- **Phiên bản: Tích hợp trong bộ MATLAB R2026a** (hoặc các phiên bản R2024, R2025). Công cụ này liên tục được MathWorks nâng cấp thuật toán linprog để tận dụng tối đa sức mạnh phần cứng của các dòng chip mới.

Hướng dẫn sử dụng

MATLAB Optimization Toolbox cho phép người dùng mô tả bài toán dưới dạng các ma trận và vectơ toán học, sau đó sử dụng các hàm tối ưu hóa có sẵn để tìm nghiệm tối ưu. Kết quả có thể được trực quan hóa và phân tích dễ dàng nhờ khả năng xử lý dữ liệu mạnh mẽ của MATLAB.

Bước 1: Khởi động phần mềm MATLAB và mở môi trường làm việc.

Bước 2: Khai báo dữ liệu của bài toán dưới dạng ma trận và vectơ.

Bước 3: Lựa chọn hàm tối ưu hóa phù hợp, chẳng hạn như hàm linprog đối với bài toán quy hoạch tuyến tính.

Bước 4: Thực hiện lệnh giải bài toán và chờ chương trình xử lý.

Bước 5: Phân tích nghiệm tối ưu và đánh giá kết quả thu được.

4.6 OR-Tools của Google

Giới thiệu

OR-Tools (Operations Research Tools) là thư viện tối ưu hóa mã nguồn mở được phát triển bởi Google nhằm hỗ trợ giải quyết các bài toán nghiên cứu tác nghiệp và tối ưu hóa trong thực tế. Đây là một trong những công cụ hiện đại và mạnh mẽ được sử dụng rộng rãi trong lĩnh vực logistics, vận tải, lập lịch và quản lý chuỗi cung ứng.

Khả năng giải

- Quy hoạch tuyến tính (LP)
- Quy hoạch hỗn hợp nguyên (MILP)
- Bài toán định tuyến phương tiện
- Bài toán lập lịch
- Bài toán luồng mạng

Ưu điểm

OR-Tools là một công cụ tối ưu hóa hoàn toàn miễn phí và mã nguồn mở, giúp người dùng dễ dàng tiếp cận và triển khai trong các dự án thực tế. Phần mềm có hiệu năng cao trong việc giải nhiều dạng bài toán tối ưu hóa và tích hợp tốt với các ngôn ngữ lập trình, đặc biệt là Python, từ đó hỗ trợ hiệu quả cho việc phát triển các ứng dụng nghiên cứu và công nghiệp.

Nhược điểm

Việc sử dụng OR-Tools đòi hỏi người dùng phải có kiến thức về lập trình để xây dựng và triển khai mô hình tối ưu hóa. Bên cạnh đó, phần mềm không cung cấp giao diện đồ họa trực quan như LINGO, khiến quá trình tiếp cận ban đầu có thể gặp khó khăn đối với người mới sử dụng.

Phiên bản hiện hành

- **Phiên bản: Google OR-Tools v9.8** (hoặc v9.9/v10.0 tùy thuộc vào các bản cập nhật mới nhất trên kho lưu trữ GitHub của Google).

Hướng dẫn sử dụng

Người dùng xây dựng mô hình thông qua các ngôn ngữ lập trình như Python, C++, Java hoặc C. Sau khi khai báo các biến, hàm mục tiêu và các ràng buộc, bộ giải của OR-Tools sẽ thực hiện tối ưu hóa và cung cấp nghiệm tối ưu của bài toán.

Bước 1: Cài đặt thư viện OR-Tools trong môi trường lập trình.

Bước 2: Khởi tạo bộ giải phù hợp với dạng bài toán cần giải.

Bước 3: Khai báo các biến quyết định, hàm mục tiêu và hệ thống ràng buộc.

Bước 4: Thực hiện lệnh tối ưu hóa bằng bộ giải của OR-Tools.

Bước 5: Thu thập, phân tích và đánh giá kết quả tối ưu thu được từ chương trình.

5. Tài liệu tham khảo

[1] GS. TSKH. Phan Quốc Khánh – TS. Trần Huệ Nương (2003), Quy hoạch tuyến tính, Việt Nam: Nhà xuất bản Giáo dục.